

ARTIFICIAL NEURAL NETWORKS

COURSE PROJECT ON

IDENTIFICATION OF OBJECTS

FROM IMAGES

MOHAMMED TARIQ S. – PES1201700961

SAHIL F. – PES1201701653

BHUVAN M. R. – PES1201701823

PROBLEM STATEMENT

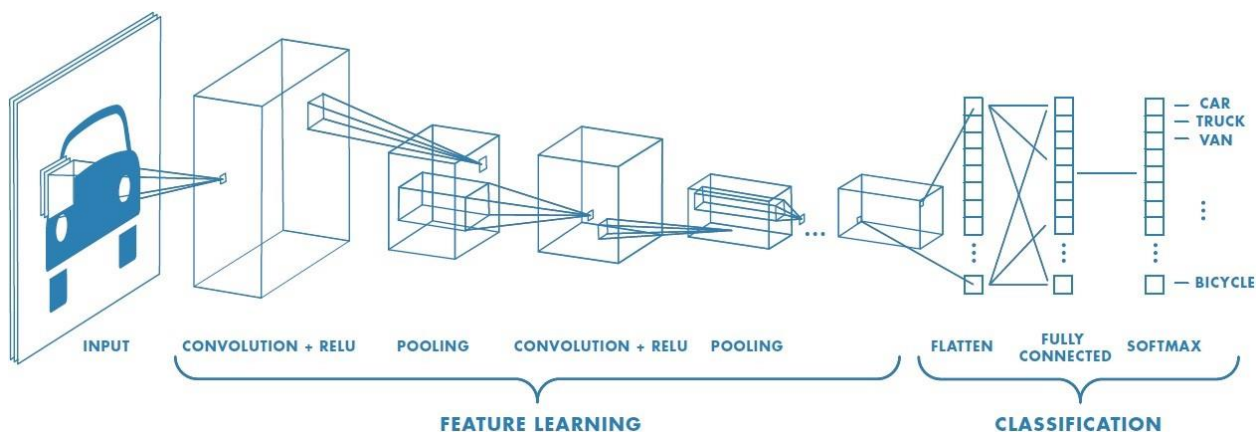
Classification of objects from the given images using feed forward Convolution Neural Network with Back Propagation algorithm.

CONVOLUTIONAL NEURAL NETWORKS

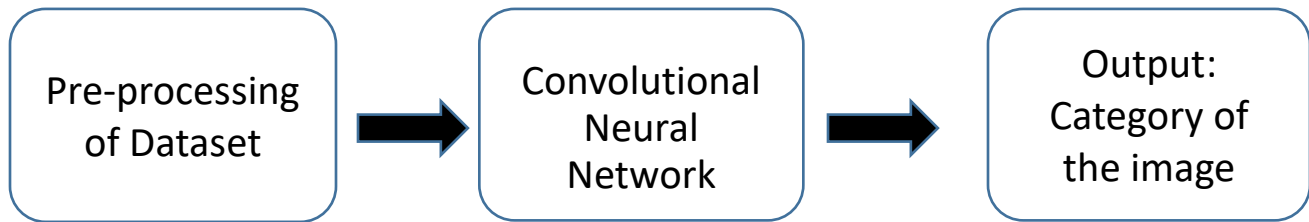
In neural networks, Convolutional neural network is one of the main categories to perform image recognition and image classification. Objects detection, face recognition, etc. are some of the areas where CNNs are widely used.

CNN image classifications takes an input image, processes it and classifies it under certain categories. Computers see an input image as array of pixels which depends on the image resolution.

In deep learning CNN models for training and testing, each input image will be passed through a series of convolution layers which are filter (Kernel) layers, normalization layers, pooling layers, fully connected layers and then to Softmax function to classify an object and obtain probabilistic values between 0 and 1. The following figure shows a complete flow of CNN to process an input image and classify the objects.



THE BASIC NETWORK MODEL



DATASET

The dataset consists of 5 categories of images –

- Caps
- Playing cards
- Rubik's Cubes
- Screwdrivers
- Torches

Each of these categories contain around 20 images.

This dataset is divided into two sets, one for training and the other for validating the data.

PRE-PROCESSING THE DATA

- **Resizing**

Since all the images of the dataset will be inputted into the neural network through an input layer, the images must be of same resolution and dimension (channel size). Using the **augmentedImageDatastore()** function, all the images of the dataset are resized to (227,227,3).

- **Splitting into training and validation set**

70% of the dataset is used for training and the rest for validation.

```
imshow = [227,227,3];  
[imdsTrain,imdsValidation] = splitEachLabel(imds,0.7,'randomized');  
augTrain = augmentedImageDatastore(imsize,imdsTrain);  
augVal = augmentedImageDatastore(imsize,imdsValidation);
```

ABOUT THE NETWORK

- **Image Input Layer**

An `imageInputLayer` is where you specify the image size, which, in this case, is 227 X 227 X 3. These numbers correspond to the height, width, and the channel size.

- **Convolutional Layer**

In the convolutional layer, the first argument is `filterSize`, which is the height and width of the filters the training function uses while scanning along the images. In this example, the number 3 indicates that the filter size is 3-by-3. You can specify different sizes for the height and width of the filter. The second argument is the number of filters, `numFilters`, which is the number of neurons that connect to the same region of the input.

- **Batch Normalization Layer:**

Batch normalization layers normalize the activations and gradients propagating through a network, making network training an easier optimization problem. Use batch normalization layers between convolutional layers and nonlinearities, such as ReLU layers, to speed up network training and reduce the sensitivity to network initialization. Use `batchNormalizationLayer` to create a batch normalization layer.

- **Relu Layer:**

The batch normalization layer is followed by a nonlinear activation function. The most common activation function is the rectified linear unit (ReLU). Use `reluLayer` to create a ReLU layer.

- **Max Pooling Layer:**

Convolutional layers (with activation functions) are sometimes followed by a down-sampling operation that reduces the spatial size of the feature map and removes redundant spatial information. Down-sampling makes it possible to increase the number of filters in deeper convolutional layers without increasing the required amount of computation per layer. One way of down-sampling is using a max pooling, which you create using `maxPooling2dLayer`.

- **Fully Connected Layer:**

The convolutional and down-sampling layers are followed by one or more fully connected layers. As its name suggests, a fully connected layer is a layer in which the neurons connect to all the neurons in the preceding layer. This layer combines all the features learned by the previous layers across the image to identify the larger patterns. The last fully connected layer combines the features to classify the images. Therefore, the OutputSize parameter in the last fully connected layer is equal to the number of classes in the target data.

- **Softmax Layer:**

The softmax activation function normalizes the output of the fully connected layer. The output of the softmax layer consists of positive numbers that sum to one, which can then be used as classification probabilities by the classification layer. Create a softmax layer using the softmaxLayer function after the last fully connected layer.

- **Classification Layer:**

The final layer is the classification layer. This layer uses the probabilities returned by the softmax activation function for each input to assign the input to one of the mutually exclusive classes and compute the loss. To create a classification layer, use classificationLayer.

TRAINING THE NETWORK

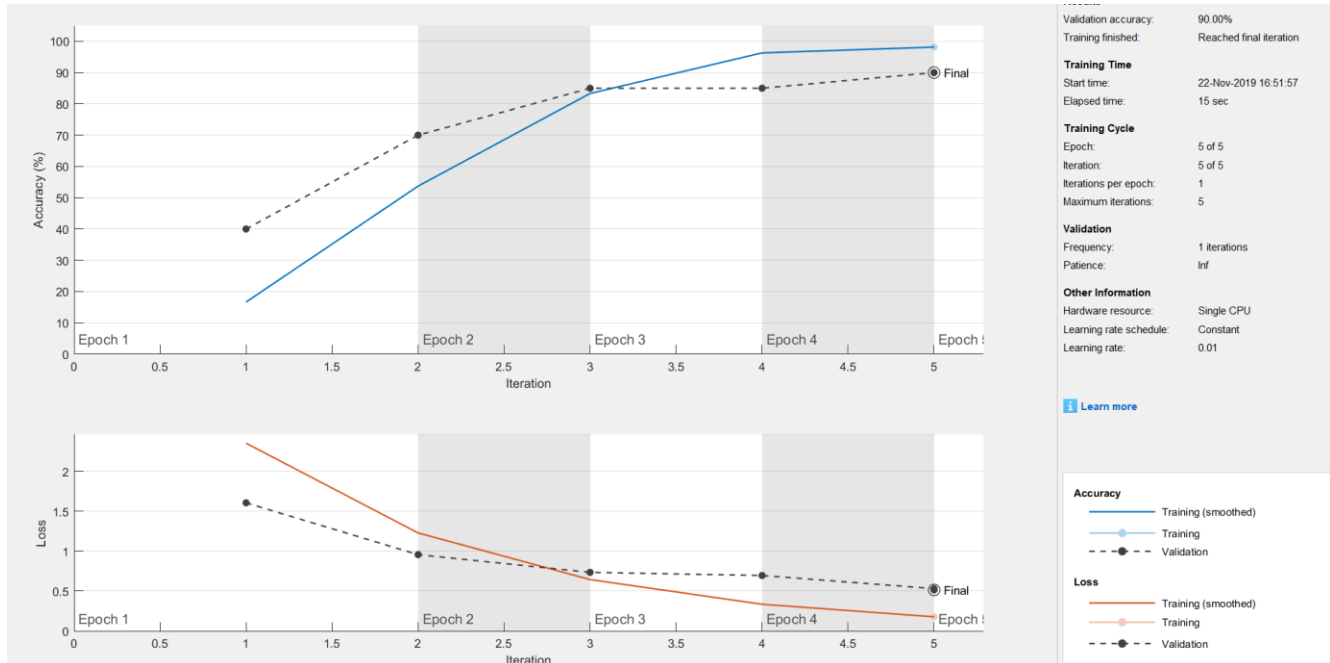
```
options = trainingOptions('sgdm', ...  
    'InitialLearnRate',0.01, ...  
    'MaxEpochs',5, ...  
    'MiniBatchSize',64,...  
    'Shuffle','every-epoch', ...  
    'GradientThreshold',1,...  
    'ValidationData',augVal,...  
    'ValidationFrequency',1,...  
    'Verbose',true, ...  
    'VerboseFrequency',1,...  
    'Plots','training-progress');
```

```
%%  
net = trainNetwork(augTrain, layers, options);
```

%%

```
[YPred,scores] = classify(net,augVal);|
pred_acc = sum(YPred == imdsValidation.Labels)/length(YPred)
```

RESULTS OBTAINED



Epoch	Iteration	Time Elapsed (hh:mm:ss)	Mini-batch Accuracy	Validation Accuracy	Mini-batch Loss	Validation Loss	Base Learning Rate
1	1	00:00:03	16.67%	40.00%	2.3516	1.6052	0.0100
2	2	00:00:06	53.70%	70.00%	1.2283	0.9566	0.0100
3	3	00:00:09	83.33%	85.00%	0.6445	0.7346	0.0100
4	4	00:00:12	96.30%	85.00%	0.3357	0.6955	0.0100
5	5	00:00:15	98.15%	90.00%	0.1785	0.5313	0.0100

pred_acc =

0.9000

TESTING THE NETWORK



YPred =

categorical

Cube

scores =

1×5 single row vector

0.0799 0.0007 0.9175 0.0000 0.0018

APPLICATIONS

1. Real time object detection and classification.
2. Self - driving cars.